

API Testing Made Simple

API (Application Programming Interface) testing is checking how well the interaction between different software components works. APIs allow different systems or software applications to communicate with each other. In API testing, you check if the data sent to or received from an API is correct, and if it follows the expected format and rules.

Common API Methods

APIs usually work with HTTP requests, and the most common HTTP methods are:

- **GET**: Retrieve data from the server.
- **POST**: Send data to the server to create something new.
- **PUT**: Update an existing resource on the server.
- **DELETE**: Remove a resource from the server.

API Testing Example with ReqRes

We will use the **https://reqres.in/** API, which is a free API for testing. This API allows us to perform operations like **GET**, **POST**, **PUT**, and **DELETE**.

1. GET: Retrieve Data

GET Request Example: Fetching user details:

`GET https://reqres.in/api/users/2`

This fetches the data for a user with ID 2. You will see details like the first and last name, email, etc.

2. POST: Create New Data

POST Request Example: Creating a new user:

`POST https://reqres.in/api/users`

You need to send data to the API. Here's an example of the JSON (Java Script Object Notation) data to send:

```
{
  "name": "John",
  "job": "Developer"
}
```

The API will create a new user with the name as "John" and job as "Developer".

3. PUT: Update Data

PUT Request Example: Updating user details:

`PUT https://reqres.in/api/users/2`

Example data to update:

```
{
  "name": "John Updated",
  "job": "Senior Developer"
}
```

The API will update the current user with the new name as "John Updated" and job as "Senior Developer".

4. DELETE: Remove Data

****DELETE Request Example**:** Deleting a user:

'DELETE https://reqres.in/api/users/2'

This will remove the user with ID 2.

Certainly! Here's the difference between **PATCH** and **PUT** requests:

5. Patch: Partial Data Update

****PATCH Request Example**:** Updating user details:

'PATCH https://reqres.in/api/users/2'

Example data to update:

```
{
  "name": "John",
  "job": "Senior Java Developer"
}
```

The API will partially update the current user with the new job as "Senior Java Developer" and keep name as same.

6. Difference between Patch and Put

PUT Request:

- **Purpose:** Used to update an existing resource or create a resource if it does not exist.
- **Behaviour:** When you send a PUT request, you are typically sending the **entire resource**, even if you want to update only part of it. If the resource does not exist, it can be created.
- **Idempotency:** PUT is idempotent, meaning that making the same PUT request multiple times will result in the same outcome. It will overwrite the resource every time, no matter how many times you call it.
- **Use case:** Use PUT when you want to replace the entire resource or ensure the resource is fully updated.

Example:

- Suppose you have a user object with the following fields: name, email, and job. If you use PUT to update the user, you would send the complete object.

```
{  
  "name": "John",  
  "email": "john.doe@example.com",  
  "job": "Developer"  
}
```

PATCH Request:

- **Purpose:** Used for partial updates to an existing resource.
- **Behaviour:** A PATCH request only sends the **fields you want to update**, not the entire resource. It's typically used to update a small part of a resource without affecting the rest.
- **Idempotency:** PATCH is not necessarily idempotent. In some cases, making the same PATCH request multiple times can produce different results, especially if the update is based on a dynamic condition.
- **Use case:** Use PATCH when you only need to update a specific field or part of a resource without replacing the entire resource.

Example:

- If you only want to update the job field for the user, you would send a PATCH request like this:

```
{  
  "job": "Senior Developer"  
}
```

Summary:

- **PUT:** Full replacement of the resource (or create it if it doesn't exist).
- **PATCH:** Partial update, only modifying the fields that are specified.

Common HTTP Status Codes and What They Mean

When you make API requests, you will get an HTTP status code in response. These codes tell you whether the request was successful or if there was an error.

1. ****200 OK**** - ****What it means****: In API testing, this is the golden response! When you send a valid request to the server (like asking for user details), a ****200 OK**** response means the server understood the request and returned the expected data.

- Example: You use an API to fetch a list of users. If everything works fine, you'll get a 200 status code along with the data you're asking for, like this:

```
{  
  
  "data": {  
    "id": 2,  
    "email": "janet.weaver@reqres.in",  
    "first_name": "Janet",  
    "last_name": "Weaver",  
    "avatar": https://reqres.in/img/faces/2-image.jpg  
  },  
  
  "support": {  
    "url": "https://contentcaddy.io?utm\_source=reqres&utm\_medium=json&utm\_campaign=referral",  
    "text": "Tired of writing endless social media content? Let Content Caddy generate it for you."  
  }  
}
```

- In Testing: If you don't get this response for valid requests, something's wrong!

2. ****201 Created****

- ****What it means****: This status is for API testing when you create a resource on the server. If you send a ****POST**** request to create something new, and it is successful, you'll get a ****201 Created**** response.

Example: You send a request to add a new user:

```
POST /users  
{  
  "name": "John Doe",  
  "email": "john.doe@example.com"  
}
```

The server responds with ****201 Created****, meaning the user is successfully added to the system.

3. ****400 Bad Request****

- ****What it means****: When you are testing APIs, a ****400 Bad Request**** response means that there was an issue with the request. This might happen if your request is missing important information, or the data format is wrong.

- Example: You try to add a user, but forget to provide the email address. The server will send a ****400**** error because the request is incomplete.

```
POST /users
{
  "name": "John"
}
```

The server can't process this without an email, so you get a ****400 Bad Request****.

4. ****401 Unauthorized****

- ****What it means****: In API testing, this means you are trying to access a resource that requires authentication, but you have not provided valid credentials (like an API key or login information).

- Example: You want to get user details, but the API requires an authentication token. If you try to access it without one, the server will respond with ****401 Unauthorized****.

```
GET /users/1
```

Without the proper token, the server rejects the request.

5. ****404 Not Found****

- ****What it means****: In API testing, a ****404 Not Found**** happens when you request a resource that does not exist. It's like searching for something on the internet, but it's not there.

- Example: You try to get a user with an ID that doesn't exist.

```
GET /users/99999
```

The server can't find a user with that ID, so you get a ****404****.

6. ****500 Internal Server Error****

- ****What it means****: A ****500**** error is something you hope you never see while testing, but it can happen if **there is** an issue on the server side. This means something broke, and it **was not** your fault.

- Example: The server is overloaded, or **there is** a bug in the backend code, and it **cannot** process your request. In this case, **you will** get a ****500 Internal Server Error****

7. ****501 Not Implemented****

- ****What it means****: When you send a request with a method (like ****PATCH**** or ****DELETE****) that the server doesn't recognize, you'll get a ****501****. The server is saying, "I don't know what you're asking me to do."

- Example: You send a ****PATCH**** request to update a user's info, but the server isn't set up to handle it. `PATCH /users/1`

```
{ "email": "new.email@example.com" }
```

Since the server doesn't know how to handle ****PATCH**** (maybe it's only configured for ****PUT****), you'll get a ****501****.

8. ****502 Bad Gateway****

- ****What it means****: When you're testing an API, sometimes the server acts as a "middleman" between you and another server. If it can't reach the other server (maybe it's down or busy), you'll get a ****502 Bad Gateway**** error.

- **Example**: The server you're trying to reach might depend on another service to gather data. If the other service is down, the API returns a ****502****.

`GET /users/2`

The server might be unable to fetch data from a database or another service and will return a ****502 Bad Gateway****

Summary of Status Codes in API Testing:

- **200 OK**: The server gave you the expected data. Your request was perfect!
- **201 Created**: You successfully created a new resource (like adding a user).
- **400 Bad Request**: The request was wrong. Maybe something was missing or formatted incorrectly.
- **401 Unauthorized**: You didn't provide the correct credentials or authorization token.
- **404 Not Found**: The resource you're asking for doesn't exist.
- **500 Internal Server Error**: Something broke on the server's side; it's not your fault.
- **501 Not Implemented**: The server doesn't recognize or support the request method.
- **502 Bad Gateway**: The server couldn't get a valid response from another server it depends on.

Authorization Types and What is Token?

Some APIs require authentication. You need to provide an authorization token or credentials to access protected resources. Here are the common types of authorization you might encounter:

- **Basic Authentication**: The simplest form of authentication. It sends the username and password encoded in the request header.
- **Bearer Token Authentication**: Uses a token (usually provided after login) to authorize the user. It's commonly used in modern APIs.
- **OAuth 2.0**: A more complex authorization mechanism where access tokens are used to grant permission to access resources without exposing user credentials.

A token is like a special pass or key that allows you to access something safely, like a door key that only works for certain people.

In the world of APIs and online services, a **token** is used to prove that you are allowed to access a particular resource or service. It's a piece of information that confirms your identity or gives permission for certain actions.

For example, when you log in to a website or an app, the server gives you a **token** (usually a long, random string of characters) that you can use every time you want to make a request (like fetching data or updating something). Instead of asking for your username and password each time, the server just checks your **token** to confirm you're allowed to do what you're asking.

Here's an analogy:

- Imagine you go to a concert and show your ticket at the entrance. Once you're in, you don't have to keep showing your ID every time you want to get into another part of the venue. Your **ticket** (or **token**) proves that you're allowed in, and the guards can just check your ticket to let you through.

Why use a token?

- **Security:** Tokens keep your sensitive information (like passwords) hidden. The server doesn't need to know your password every time.
- **Convenience:** You don't have to log in repeatedly. Once you have a token, you can use it to access resources without logging in every time.

Why Use These Authorization Methods?

- **Basic Authentication** is easy but less secure. It's suitable for internal or low-risk APIs.
- **Bearer Token Authentication** is commonly used for APIs where security is a concern but a simpler alternative to OAuth 2.0.
- **OAuth 2.0** is the most secure option, especially for APIs that require access to sensitive data.

How to Test an API

API testing can be done manually or automatically. Here's how you can test APIs using two common tools: **Postman** and **Rest Assured**.

Using Postman for Manual Testing

Postman is a simple tool to send requests to an API and check responses.

1. **Install Postman:** Download it from the official website.
2. **Create a New Request:** Choose the type of request (GET, POST, etc.), and enter the URL for the API.
3. **Send the Request:** Hit the "Send" button and see the response.
4. **Check the Response:** Verify that the status code is correct and the data matches what you expect.

Using Rest Assured for Automation

Rest Assured is a library in Java that helps automate API testing.

1. **Set up Rest Assured:** Add the dependency in your project's `pom.xml` file.
2. **Create a Test:** Write Java code to make requests and check responses.
3. **Run the Test:** Validate status codes, response times, and data.

Key Points to Remember

- Test all possible scenarios, including valid and invalid data.
- Check that the data returned matches expectations.
- Use automation tools like Rest Assured to speed up testing.
- Validate the status code, response body, and headers.
- Use proper authorization methods for secured APIs.